

杨翔皓

求职方向：测试开发工程师 / AI 测试开发工程师 / 质量工程师 / 自动化测试工程师

目标城市：杭州

期望薪资：18-25K

电话：15397483294

GitHub：https://github.com/yyXDyy

个人主页：https://yyxdyy.github.io/resume/



个人优势

- 风险洞察**：具备 ToB 云平台测试开发与质量保障经验，能够从 PRD、Wiki、issue、截图、工单和线上问题中拆解用户路径、异常状态、权限边界、计费规则和环境差异，并根据风险优先级确定深度验证、自动化回归或检查清单覆盖范围。
- 技术与 AI 工程化**：熟悉 Kubernetes/Linux 环境排障、Playwright 自动化、接口验证和 AI 辅助测试，能使用 TypeScript、Python、Shell、kubectl、Postman、JMeter、Fiddler 等工具完成 UI 回归、Pod/服务状态检查、资源清理、失败证据采集和报告输出。
- 业务与数据意识**：围绕 Sealos 多模块发版、私有化交付和用户支持提升质量保障效率，能从工单量、线上问题、监控现象和业务影响反推测试策略，结合风险、频率和维护成本调整测试投入。
- 质量内建**：注重证据留存、问题复现、缺陷回归和跨团队沟通，能将一次性测试经验沉淀为可复用的用例、脚本、检查清单、SOP 和新人带教材料，推动高风险问题前置暴露并减少同类问题重复发生。

专业技能

- 风险驱动测试设计**：需求澄清、异常路径拆解、权限/计费/兼容性边界分析、测试优先级判断、测试左移。
- Kubernetes/Linux 排障**：Pod/Service/PVC 管理，日志/事件分析，容器、镜像、网络与存储问题定位。
- 自动化测试开发**：Playwright + TypeScript, Python/Shell, UI 自动化、接口校验、Kubernetes 状态检查与证据采集。
- 接口与稳定性验证**：Postman/Fiddler 抓包，JMeter 压测，API/WebSocket 验证，可控负载与监控异常场景模拟。
- AI 辅助测试工程化**：AI 驱动需求拆解、风险分析、用例生成、可执行性审查、traceability matrix、claim boundary、SOP 沉淀。
- 业务与团队影响**：工单治理、知识库建设、质量复盘、预上线检查清单、跨团队沟通与新人带教。

工作经历

Sealos 云服务平台

测试开发工程师

2025.01 - 至今

工作内容

S | 背景

Sealos 是面向云原生场景的超级应用平台，涉及公有云、私有化交付、DevBox、数据库、应用商店、计费订阅、GPU/AI 训练、用户支持等多个模块。平台底层依赖 Kubernetes、容器、网络、存储、镜像、数据库等能力，测试对象不仅是单点功能，还包括跨模块链路、环境兼容性、权限边界、计费准确性和线上稳定性。

实际工作中的主要挑战包括：

- 业务迭代快，多个模块经常并行提测，测试任务容易被打散。

- 私有化、海外区、多 AZ、缓存切换、老用户登录态等场景复杂，单纯功能验证不足以覆盖真实风险。
- DevBox、数据库、GPU 训练、应用部署等问题往往与 Kubernetes 环境、网络、镜像、日志、资源状态有关，需要测试人员具备一定环境排查能力。
- 用户工单量较高，部分问题会直接消耗研发精力，需要通过问题分级、知识库、文档和缺陷沉淀减少重复沟通。
- 早期测试用例维护和回归规范不够稳定，需要逐步建立更体系化的测试资产和发布验证机制。

因此，工作重点不只是执行测试，还包括让问题尽量在上线前暴露、让线上问题能被快速定位，并将经验沉淀为可复用机制。

T | 任务

在 Sealos 的质量保障工作中，主要承担以下职责：

1. 负责多个核心模块的功能测试、集成测试、回归测试和发版验证，包括 DevBox、数据库、应用商店、Admin 管理面板、订阅制/费用中心、GPU 监控、私有化版本等。
2. 参与高风险变更的测试保障，例如订阅制后端重构、数据库底层升级、kb0.9 升级、计费 webhook 修复、私有化纯内网环境验证等。
3. 建设并维护测试用例、测试报告、缺陷文档和回归清单，推动测试过程从"临时执行"转向"有记录、可追溯、可复用"。
4. 结合 Playwright、Shell、Kubernetes 环境排查和 AI 辅助能力，探索自动化回归、测试工件生成和问题诊断流程。
5. 处理和分析用户工单，沉淀高频问题文档、知识库和分级响应机制，降低研发被重复问题打断的成本。
6. 带教测试/支持实习生或新同学，将可标准化的支持任务、排查步骤和文档方法逐步交接出去。
7. 在需求评审、提测前沟通、用例评审、发版后复盘等环节前置介入，推动质量问题更早被发现。
8. 从测试投入收益出发调整验证策略：高风险链路深挖，低价值或不稳定脚本谨慎自动化，把精力放在更可能影响用户、稳定性和研发效率的问题上。

A | 行动

1. 核心功能与版本测试：从单点验证转向链路覆盖

在日常测试中，不只验证页面功能是否可点击，而是围绕用户真实路径、异常状态和平台底层链路设计测试场景。对需求文档中未明确说明的老用户状态迁移、缓存切换、多 AZ 展示、私有化网络限制、资源清理失败、计费边界和权限继承等场景，会在评审阶段提前拆解。例如：

- 在 DevBox 测试中，覆盖 GitHub 导入、本地上传导入、新 runtime、新 IDE、权限提示、镜像瘦身、连接异常、SSH 插件兼容等高频路径。
- 在数据库测试中，关注创建、访问、外网开关、暂停/恢复、参数变更、API 返回格式、升级兼容等场景。
- 在订阅制和费用中心测试中，重点验证订阅创建/取消、费用展示、多 AZ 兼容、老用户登录态、缓存切换、国内外区域兼容等风险点。
- 在私有化测试中，关注部署、访问、权限、网络、模板可用性、镜像同步、纯内网环境限制等完整闭环。
- 在 GPU/AI 训练相关问题中，参与多卡训练异常复现、GPU 监控环境配置、NFS 梳理等工作，协助定位环境或框架兼容问题。

对于复杂问题，结合日志、接口返回、Kubernetes 资源状态、用户操作路径和环境差异进行分析，避免只停留在"页面表现异常"。同时根据风险影响和复现频率判断验证深度，将部分低风险或低收益场景沉淀为检查项。

2. 高风险项目保障：把事故和问题复盘成测试机制

在 2025 年 Q4 的订阅制上线过程中，国内订阅制代码兼容上线后暴露了缓存冲突和多 AZ 费用展示异常问题。参与问题定位和复盘后，将问题拆解为：

- 老用户登录态与缓存切换冲突。
- 前后端兼容不一致导致多 AZ 展示异常。
- 预上线验证对多 AZ、缓存切换、老用户状态迁移覆盖不足。
- 接口变更同步机制不够明确。

针对这些问题，推动后续测试中增加：

- 多 AZ 验证清单。
- 老用户登录态和缓存切换专项用例。
- 费用展示、计费树、订阅链路等关键检查项。
- 接口变更同步和预上线验证要求。
- 国内/海外环境差异和关键链路回归检查。

数据库底层升级过程中，虽然上线前已有测试计划和回滚方案，但仍暴露出 API 返回格式异常、Redis 恢复边界等问题。复盘后，将升级类测试从“单次项目验证”沉淀为更明确的检查方向，包括 API 兼容、暂停/恢复、外网开关、环境一致性、回滚路径等。

这些经历沉淀出一条稳定方法：测试不能只在提测后找 Bug，还要把线上问题收敛成下次发版前必须检查的门禁。

3. 测试资产建设：用例、缺陷、报告和回归清单可追溯

持续整理 Sealos 相关测试文档、用例表、缺陷记录和回归报告，逐步形成可复用的测试资产。已有测试资产包括：

- 6000+ 条带来源和验证记录的测试用例/检查项。
- 400+ 个测试文档或表格节点。
- 84 个用例表和 6 个多维表格。
- 覆盖公有云、私有化、DevBox、数据库、应用商店、Admin、订阅制、GPU 监控、用户支持等方向。

4. 自动化与 AI 测试探索：从工具尝试回到可落地流程

在自动化测试方面，使用 Playwright 编写、调试和维护自动化用例，并将其用于部分高频回归路径。针对 Admin 管理面板、应用商店、数据库等场景编写了 15 个核心自动化用例，并提交到 `labring-sigs/sealos-issues` 私有仓库中。

自动化策略以投入收益为前提：在产研节奏偏敏捷的场景下，先通过手工验证明确稳定路径，再将自动化作为回归兜底能力，优先覆盖稳定、高频、重复、风险较高的链路。对变化频繁、断言不稳定或维护成本过高的脚本，优先保留为检查清单或半自动验证，避免为了数量扩张自动化范围。

在 AI 测试方向，早期尝试发现直接生成用例与当前团队产研节奏并不完全匹配，因此后续调整为更稳妥的工程化路径：

- 先用 AI 辅助做需求拆解、测试点生成和诊断思路整理。
- 再把稳定下来的关键 Case 转成可执行测试。
- 将线上问题、缺陷复盘和高频路径沉淀为回归集。
- 逐步接入发布流程，形成冒烟、回归、门禁或强提醒机制。

该实践形成了更清晰的边界：AI 可以提高测试设计和脚本生成效率，也能辅助整理根因定位思路，但质量保障最终仍需要稳定流程、可靠用例、明确断言和可追溯证据。

5. Kubernetes、Linux 与环境问题排查：在真实问题中积累平台理解

Sealos 的测试工作天然接触 Kubernetes、容器、镜像、网络、存储、日志和资源状态。Kubernetes 相关能力主要来自私有化发版、DevBox、数据库、GPU 监控、应用部署、镜像同步和用户问题排查等真实场景。

参与或处理过的相关场景包括：

- 私有化版本发版测试和纯内网环境验证。
- Admin 管理面板镜像同步与代理测试。
- DevBox 镜像更新、连接异常、Runtime、IDE、SSH 插件兼容测试。
- GPU 监控环境配置和 NFS 梳理。
- 多卡模型训练异常复现与验证。
- 数据库升级、外网访问、API 返回格式、Redis 恢复边界问题。
- 应用部署、WebSocket 复现、镜像拉取慢、数据库上传异常等用户问题。

在这些问题中，能够结合现象、日志、配置、环境差异和用户路径进行排查，熟悉云原生平台常见问题的基础定位思路，能在 Kubernetes 和 Linux 环境下完成初步定位，并与开发协作推进复杂问题闭环。

6. 业务问题与团队协作：把重复问题变成可复用机制

在用户支持和工单分析中，关注问题背后的业务影响和研发消耗，而不是只看单个 Bug 是否关闭。例如身份认证、计费展示、应用部署、镜像拉取慢、数据库访问异常等问题，如果反复出现，会持续消耗研发和支持精力。针对这类问题，整理高频问题知识库、分级响应流程、排查模板和新人 checklist，让支持同学能先处理标准问题，让研发把精力留给真正需要代码修复或架构判断的问题。

这一阶段的工作重点从单点问题处理延伸到流程建设和团队复用。通过用例评审、预上线检查项、问题复盘、自动化回归和带教，将质量责任拆进需求、开发、测试和支持的日常流程，减少同类问题重复发生。

工作业绩

R | 结果

1. 测试资产建设方面，累计整理 6000+ 条带来源和验证记录的测试用例/检查项，覆盖 400+ 个测试文档或表格节点、84 个用例表和 6 个多维表格。
2. 2025 年 Q3，测试记录中包含 2484 条测试用例，发现 643 个 Bug，用例通过率 74.1%；2025 年 Q4，记录 1465 条测试用例，发现 308 个 Bug，通过率提升至 79%。
3. 2026 年 Q1，通过规范用例、前置沟通和引入部分自动化回归，将回归测试耗时从平均 4 小时/轮降至约 2.4 小时/轮，达到 OKR 中"效率提升 40%"的目标。
4. 2026 年 Q1，完成 17 项测试任务，覆盖私有化发版、订阅制、多卡训练异常、GPU 监控、Admin 镜像同步等高风险或关键模块。
5. 在自动化测试方面，编写 15 个核心自动化用例，覆盖 Admin 管理面板、应用商店、数据库等高频路径，并沉淀到公开 issue/测试记录中。
6. 在用户支持方面，处理和分析 Q1/Q2 大量工单：Q1 工单 1584 条，Q2 工单 1884 条。通过高频问题整理、知识库沉淀、分级响应和支持梯队建设，减少重复问题对研发的干扰。
7. 通过工单治理和支持流程优化，研发技术精力分散情况统计优化 78.6%；身份认证相关问题收敛后，相关周工单量下降约 60%。
8. 在订阅制、费用中心、数据库升级、私有化、DevBox 等高风险场景中，将线上问题和发版问题复盘为预上线验证清单、专项用例和回归检查项，推动测试从"发现问题"向"预防问题"转变。
9. 在复杂环境问题中，持续积累 Kubernetes、Linux、容器、镜像、网络、存储和日志排查经验，能够独立处理基础环境问题，并与开发协作定位更复杂的链路问题。
10. 在团队协作方面，参与支持同学和测试/支持实习生带教，整理新人 checklist、排查模板和支持文档，让部分工作从个人经验转为团队可复用流程。
11. 在测试策略上关注投入收益：稳定高频链路优先自动化，变化频繁或低价值场景保留为清单/人工审查，减少脚本维护成本，将质量保障资源集中到更可能影响用户和稳定性的场景。

项目经历

1. Sealos 云服务平台

项目链接：<https://hzh.sealos.run>

代码/记录：<https://github.com/labring-sigs/sealos-issues>

项目描述：

- 建设 Sealos 多模块测试记录与自动化资产仓库，覆盖 DevBox、DataFlow、AppStore、CronJob、数据库、GPU、对象存储和 Admin 等模块，用于沉淀缺陷、用例、脚本和验证报告。
- 使用 Playwright + TypeScript 编写真实 UI 回归脚本，覆盖 Desktop 登录、iframe 应用进入、数据库列表/连接树/表详情、CronJob 窗口、应用商店模板部署等高频路径。
- 将 UI 操作与 Kubernetes 状态检查结合，校验 Pod Running/Ready、kubectl exec、SSH runtime、公网入口、端口连通性和资源清理，解决"页面成功但底层资源不可用"的验证盲区。

- 开发 AppStore 模板批量创建与验证工具，支持并发窗口、Ready 判断、UI 卸载、kubectI 兜底清理、失败证据采集和 Markdown/JSON 报告输出。
- 将 CronJob 飞书用例资产转为 Playwright 测试矩阵，保留 242 条用例的 traceability，并按 smoke、readonly、mutating、semantic audit 等模式区分执行边界，控制自动化维护成本。

项目成绩：

针对 Sealos 多模块回归依赖人工操作、底层资源状态难以快速确认、失败证据分散的问题，建设测试记录与自动化资产仓库，将真实 UI 操作、Kubernetes 状态检查和报告证据沉淀到同一套工具链中。

2. Customer Advocate Agent（黑盒测试 Agent）

项目链接：<https://github.com/yyXDyy/Customer-Advocate-Agent>

项目描述：

- 设计面向 PRD、Wiki、截图、API spec 和 issue 的 AI 黑盒测试流程，将需求接入、风险分析、澄清冻结、产品观察、用例生成、可执行性审查、自动化规划和交付审计拆成 Stage 0-9，解决"需求未澄清就直接出用例"的问题。
- 将测试任务拆分为 requirement analyst、risk strategist、product observer、case architect、executability reviewer、automation planner、delivery auditor 等 11 个协作 Skill，让 AI 输出从一次性生成改为分阶段审查。
- 设计 requirement SSOT、risk report、freeze decision、traceability matrix、case source、claim boundary 等产物，明确每个测试结论的来源、适用范围和待人工确认点，减少遗漏和过度推断。
- 建立"AI 负责语义判断、脚本负责机械校验"的边界，脚本只校验字段、格式、路径和证据策略，避免把格式校验或 AI 推测误写成"测试通过"。这套流程的重点不是让 AI 替代测试判断，而是让需求拆解、风险分析、用例生成和根因诊断有更稳定的工程边界。

项目成绩：

针对 AI 直接生成测试用例时容易遗漏需求、臆造预期结果、过度声明测试结论的问题，设计一套有阶段门禁和证据边界的黑盒测试工作流。

3. Monitor-Test 可观测性场景模拟器

项目链接：<https://github.com/yyXDyy/Monitor-Test>

项目描述：

- 通过环境变量定义 CPU、内存、GPU、日志和 DB 负载参数，支持 steady、spike、step、jitter、random、outlier、OOM 等场景，解决监控测试只能依赖偶发线上负载的问题。
- 将工具拆分为 config、scenario、runtime、orchestrator 和 modules，分别负责参数解析、负载曲线生成、模块生命周期、心跳日志和退出控制，便于扩展新的负载类型。
- 使用可控负载验证监控曲线、异常点展示和高频刷新稳定性，并补充单元测试覆盖解析、场景生成和运行控制逻辑，减少工具自身改动引入回归问题。这个项目更偏稳定性验证思路：用可复现的异常输入提前发现监控展示和刷新链路里的隐患。

项目业绩：

针对云平台监控功能缺少稳定、可复现测试负载的问题，开发资源负载模拟器，用可控输入验证监控曲线、异常点展示和刷新稳定性。

其他测试经历

- 面向客户支持场景编写 Paddle X 训练使用教程，整理环境准备、数据校验、训练、评估、推理和资源限制处理等步骤。
- 参与 Web、App、小程序自动化测试，以及接口测试、性能测试、弱网测试实践。

教育经历

集美大学

本科，计算机科学与技术

荣誉与证书

- 福建省软件测试省赛三等奖。
- 院系一等奖学金、院系三等奖学金。
- 大学英语四级。
- 1+X Web 应用软件测试（中级）。
- 1+X 计算机视觉应用开发（初级）。